CS-354: Complier Construction

(Spring 2026)

Lecture 4

Adnan Rashid

Email: adnan.rashid@seecs.edu.pk

Web: https://adrashid.github.io/

Recap

- Structure of Compilers
- Types of Compilers
  - Two Pass

Outline

- Types of Compilers
  - Two Pass

Lecture's Content

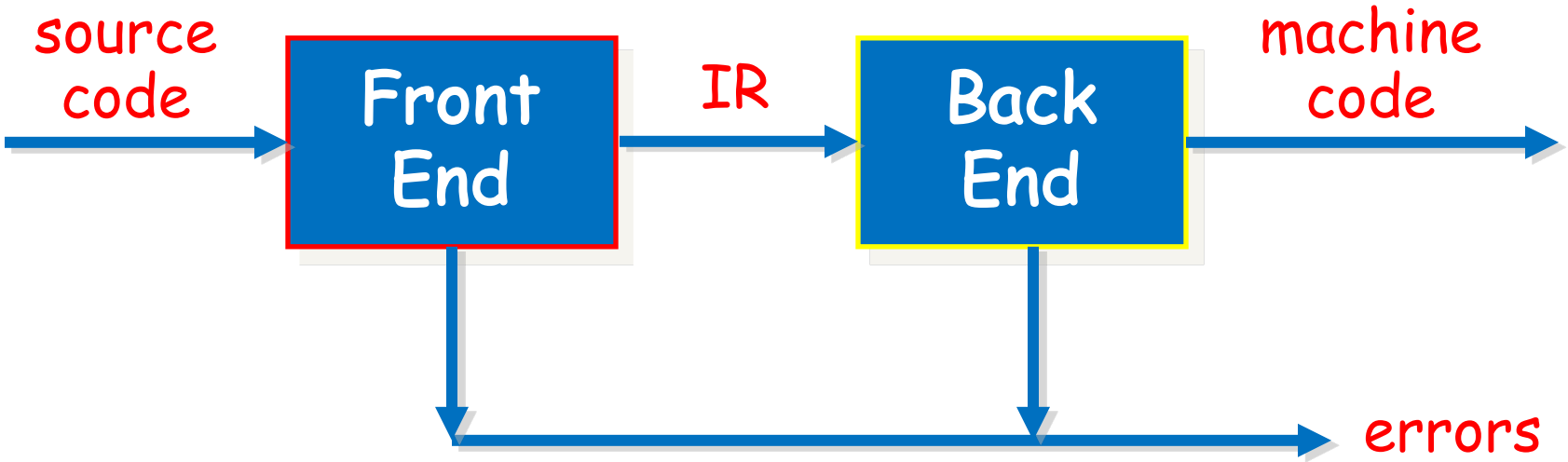
Compiler: Principles, Techniques and Tools (2nd Edition)

by

Alfred V. Aho et al.

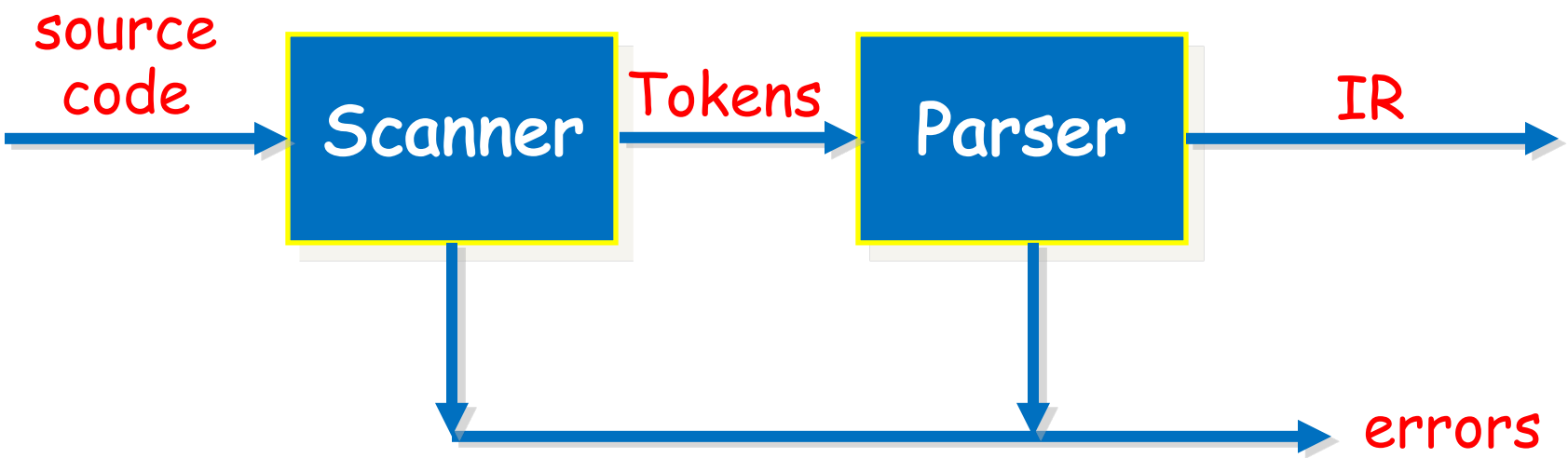
Sections 1.1 and 1.2 (Page 3 onwards)

Two Pass Compiler



- Recognizes legal (and illegal) programs
- Report errors in a useful way
- Produce IR and preliminary storage map

The Front End



Modules

- Scanner (Lexical Analyzer): Maps character stream into words – basic unit of syntax
- Parser: Recognizes context-free syntax and reports errors

Scanner (Lexical Analyzer)

- Maps character stream into words – basic unit of syntax
- Produces pairs
  - a word and
  - its part of speech
- Example

  x = x + y

  becomes

  <id,x>
  <assign,=>
  <id,x>
  <op,+>
  <id,y>

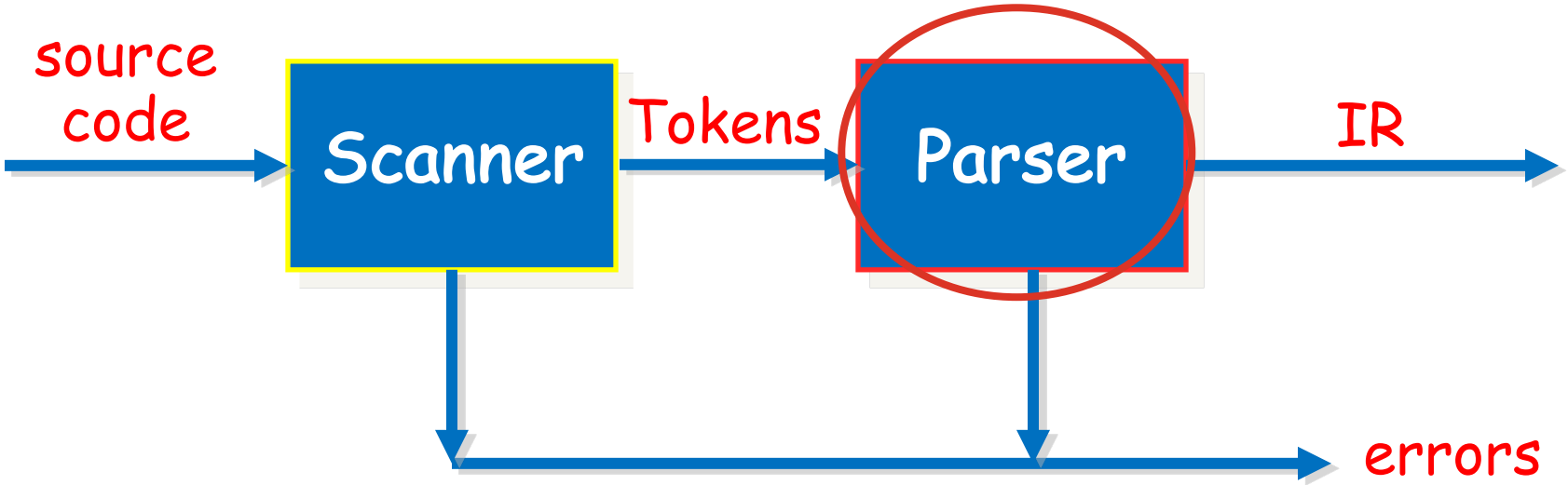
<id,x>

token type

attribute value

We call the pair:
<token type, value> a token
Typical tokens: number, identifier, +, -, new, while, if

The Front End



- Recognizes context-free syntax and reports errors
- Guides context-sensitive (semantic) analysis
- Builds IR for source program

Context Free Grammar (CFG)

Context-free syntax is specified using a Context-free Grammar (CFG) = (S,N,T,P)

- S is the start symbol
- N is a set of non-terminal symbols
- T is set of terminal symbols or words
- P is a set of productions or rewrite rules

Example

Productions

Sequence of terminals and non-terminals (variables)

Grammar:

$S \rightarrow aSb$

$S \rightarrow \lambda$

Terminals

Non-terminals

The right side may be λ

Example: Derivation

Grammar

$S \rightarrow aSb$

$S \rightarrow \lambda$

Derivation of string: ab

$$S \Rightarrow aSb \Rightarrow ab$$

$S \rightarrow aSb$ $\qquad$ $S \rightarrow \lambda$

Activity: Derivation

Grammar

$S \rightarrow aSb$

$S \rightarrow \lambda$

Derivation of string: aabb

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

$S \rightarrow aSb$ $\qquad$ $S \rightarrow aSb$ $\qquad$ $S \rightarrow \lambda$

Example: Language of the Grammar

Grammar

$S \rightarrow aSb$

$S \rightarrow \lambda$

Language of the Grammar:

$$L = \{a^n b^n : n \geq 0\}$$

Context Free Grammar (CFG)

Grammar for expressions

1. goal → expr
2. expr → expr op term
3. | term
4. term → <u>number</u>
5. | <u>id</u>
6. op → +
7. | -

For this CFG

S = goal

N = {goal, expr, term, op}

T = {number, id, +, -}

P = {1, 2, 3, 4, 5, 6, 7}

left side wale are variables, rest are terminals

Example: Derivation

- Given a CFG, we can derive sentences by repeated substitution
- Consider the sentence (expression):

  x + 2 – y

Grammar for expressions

1. goal → expr
2. expr → expr op term
3. | term
4. term → <u>number</u>
5. | <u>id</u>
6. op → +
7. | -

Production	Result
	goal
1	expr
2	expr op term
5	expr op y
7	expr - y
2	expr op term - y
4	expr op 2 - y
6	expr + 2 - y
3	Term + 2 - y
5	x + 2 – y

Parsing

- To recognize a valid sentence in some CFG, we reverse this process and build up a parse
- A parse can be represented by a tree: parse tree or syntax tree

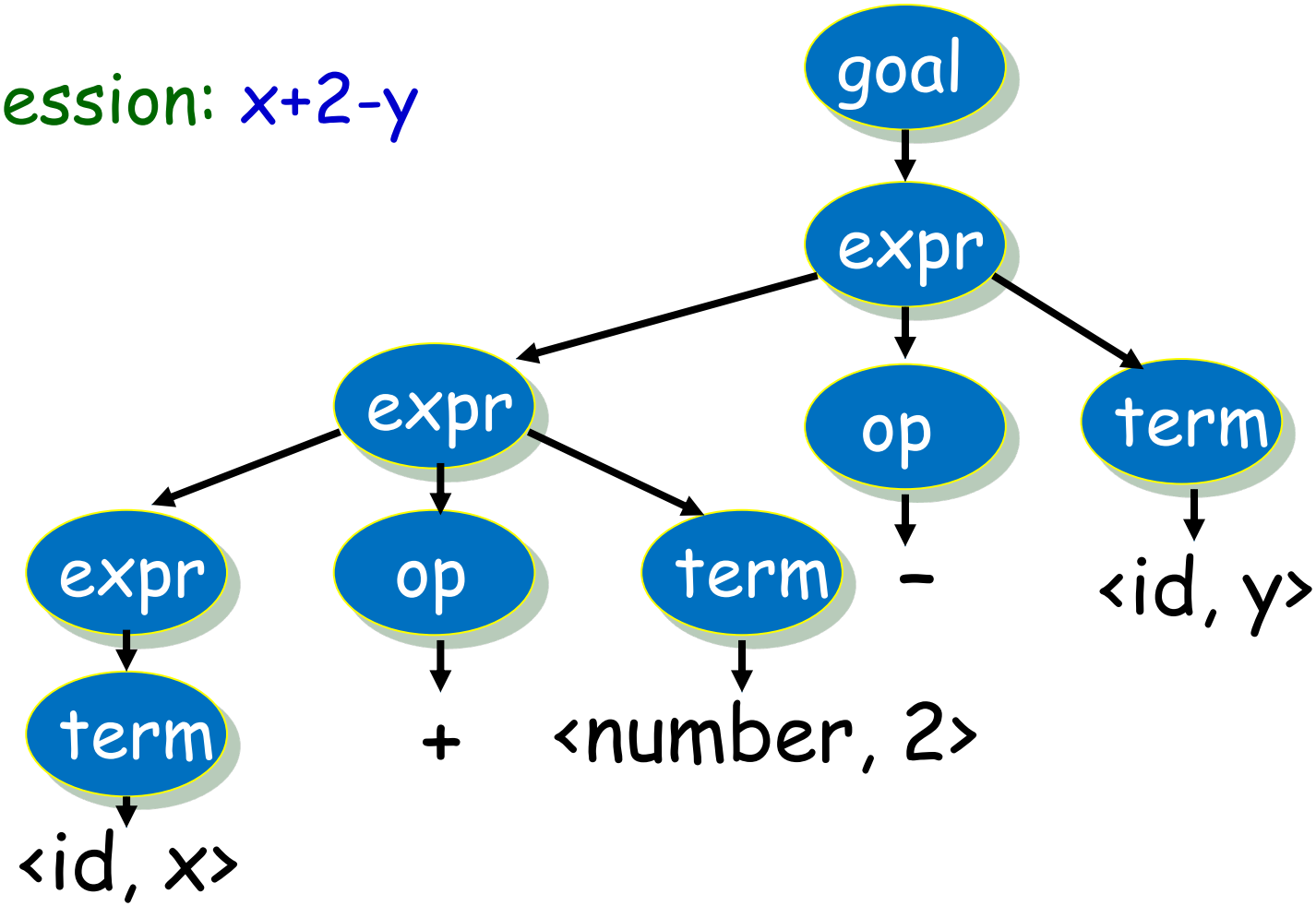
Grammar for expressions

1. goal → expr
2. expr → expr op term
3. | term
4. term → number
5. | id
6. op → +
7. | -

Production	Result
	goal
1	expr
2	expr op term
5	expr op y
7	expr - y
2	expr op term - y
4	expr op 2 - y
6	expr + 2 - y
3	Term + 2 - y
5	x + 2 – y

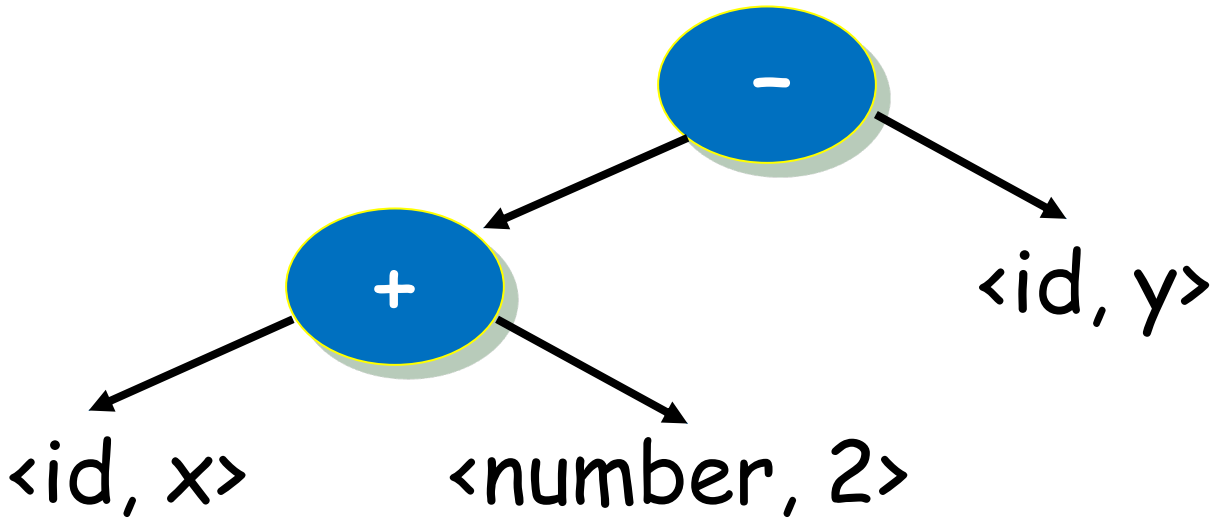
Syntax Tree (aka Parse Tree)

Expression: x+2-y



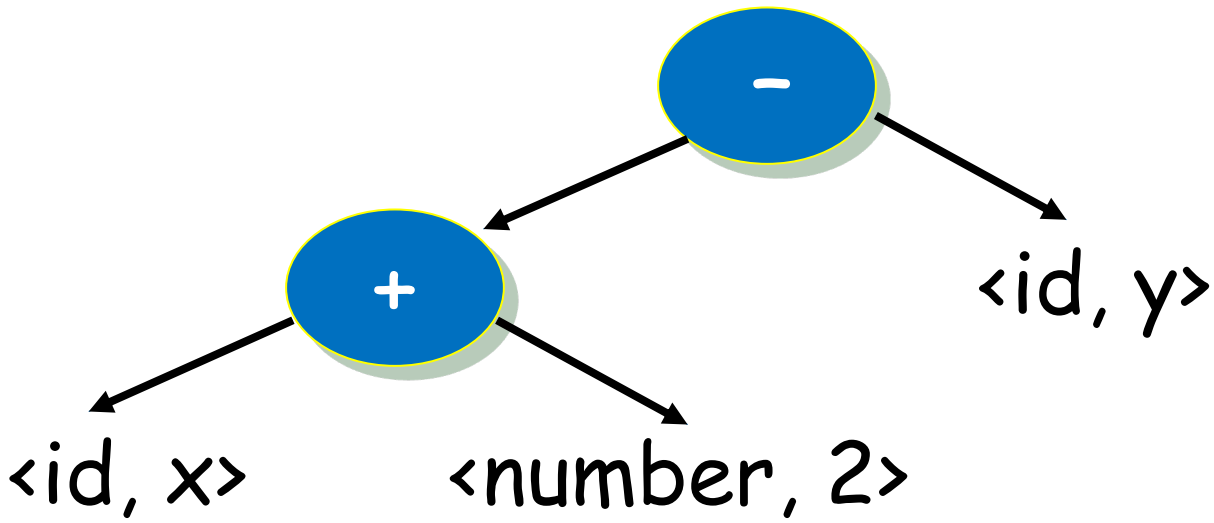
Abstract Syntax Tree (AST)

- The parse tree contains a lot of useless information
- Compilers often use an abstract syntax tree (AST)



Abstract Syntax Tree (AST)

- An AST is a much more concise representation



- It summarizes the grammatical structure without any details of derivation
- ASTs are one kind of intermediate representation (IR)

Thank you!